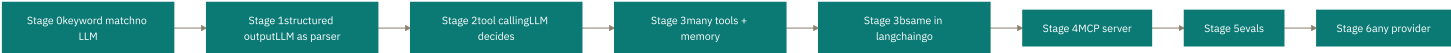


A GUIDED TOUR OF THE REPO

Build an agent from scratch, one idea at a time

agent-lab is a small Go project that asks for a joke in plain English and fetches one from a real API. The domain is tiny on purpose, so each stage can teach a single idea about LLM apps without anything else in the way.



Every stage is built and tested offline. Stage 6's OpenAI and Claude adapters have not run against the live APIs.

Who is in charge at each stage?

The whole project is really one dial: how much control moves from your code to the model.

STAGE	INTERPRETS THE REQUEST	CHOOSES THE NEXT ACTION	RETRIES ON FAILURE	REMEM
0 Keywords	your code	your code	☐	☐
1 Structured	the model	your code	☐	☐
2 Tool loop	the model	the model	☒	☐
3 Multi-tool	the model	the model	☒	☒
3b Framework	the model	the framework's loop	☒	☒
4 MCP server	the client's model	the client	decided by whichever client con	
6 Any provider	the model	the model	☒	☒

The one rule that runs through every stage

The joke client takes a typed `jokeapi.Request`, never free text.

The predecessor project had the model write `category=misc&blacklist=racist` as a string, and a keyword matcher re-parsed it. The blacklist was silently dropped. Here the model fills in a typed form, our code runs it through `Normalize()`, and only then does anything touch the network.

Repository map

```
cmd/jokes/          REPL; picks a stage with -stage
internal/jokeapi/    typed JokeAPI client + the Request schema
internal/dadjoke/    icanhazdadjoke.com client (second joke source)
internal/ollama/     raw-HTTP Ollama chat client with memory limits
internal/stage0/     keyword interpreter
internal/stage1/     structured-output interpreter (ollama.go, claude.go)
internal/stage2/     tool-calling agent loop
internal/stage3/     multi-tool agent with memory (tools.go = tool registry)
internal/stage3lc/   stage 3 in langchaingo
internal/stage4/     MCP server (hand-written JSON-RPC) + the shared joke tools
internal/evals/      eval cases, checks and runner
internal/provider/   Ollama, OpenAI and Claude behind one interface
internal/stage6/     stage 3's agent over provider.Provider
internal/trace/      step printer behind -trace
cmd/jokes-mcp/       stage 4 server on stdio
cmd/jokes-eval/      stage 5 eval runner
```

Plain HTTP, no SDK

The Ollama, OpenAI and Claude clients are `net/http` plus structs that mirror each wire format. The one framework is stage 3b, because that is its lesson.

Duplication is intentional

Stages are meant to be read side by side, so the agent loop is copied between stages rather than shared. The exception is the joke tools: stage 6 reuses stage 4's.

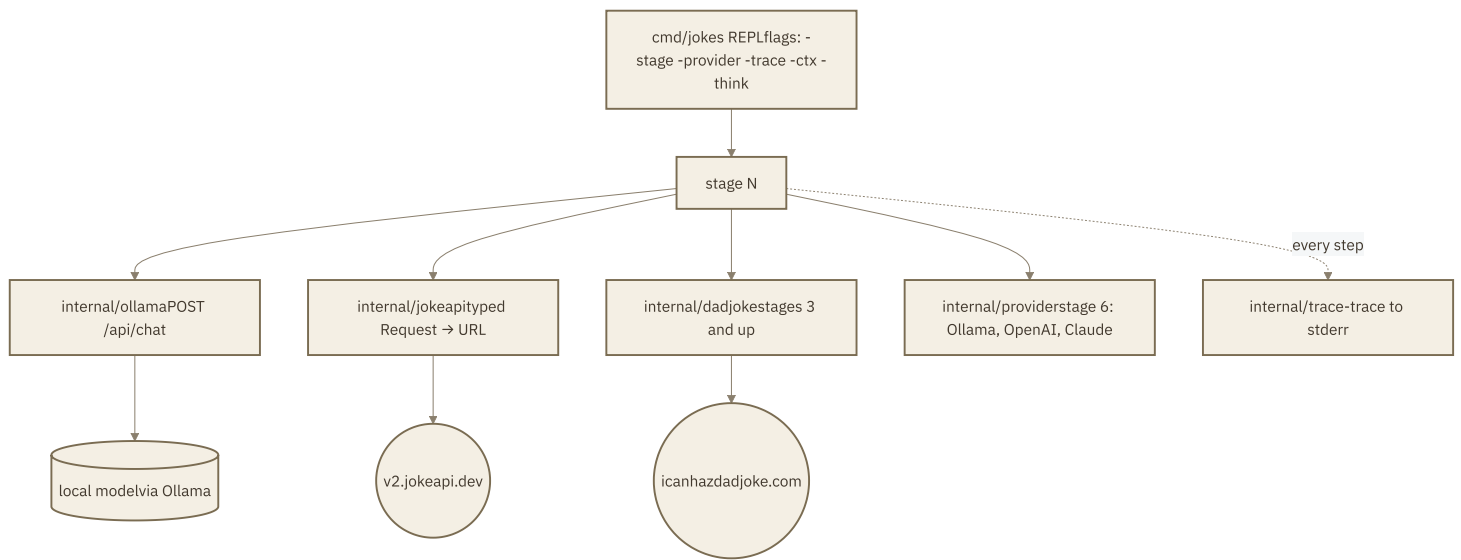
Everything is tested offline

Every API and model is faked with `httptest`. `make test` needs no network. `make live` hits the real joke APIs.

Local and frugal

Default model `qwen3.5:9b-m1x` through Ollama on a 16 GB Mac, with context 8192, output cap 2048 and a 2 minute keep-alive.

The shared plumbing

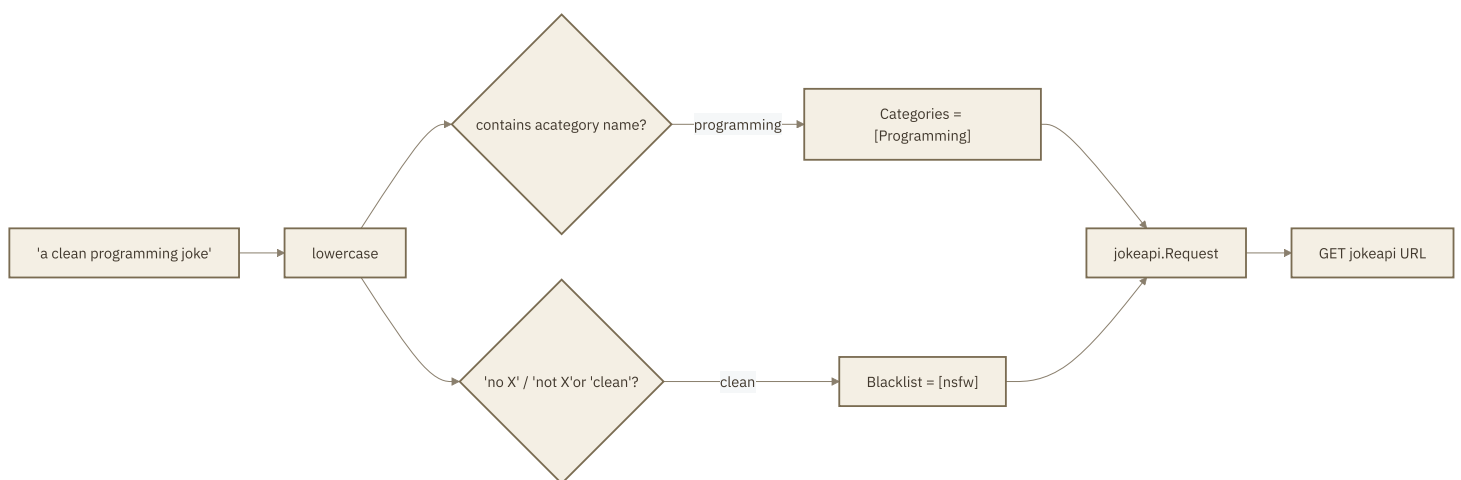


The REPL builds one stage, and every stage reuses the same clients and the tracer.

● STAGE 0 · INTERNAL/STAGE0 · **BUILT**

Keywords: the baseline to beat

Before adding an LLM, solve the problem without one. `stage0.Interpreter` lowercases your text and looks for known words: category names, "twopart" or "one-liner", and the pattern `no <flag>` or `not <flag>`.



Everything here is `strings.Contains`. It is fast, free and predictable.

Where it works

"programming joke", "a two-part pun", "no politics".
Exact words in, exact filters out.

Where it breaks

"keep it family friendly" matches nothing. "clean" maps only to `nsfw`, not the full blacklist. "nothing

about religion" slips past no religious .

LESSON

A baseline makes every later stage measurable. If the LLM can't beat `strings.Contains`, it isn't earning its cost.

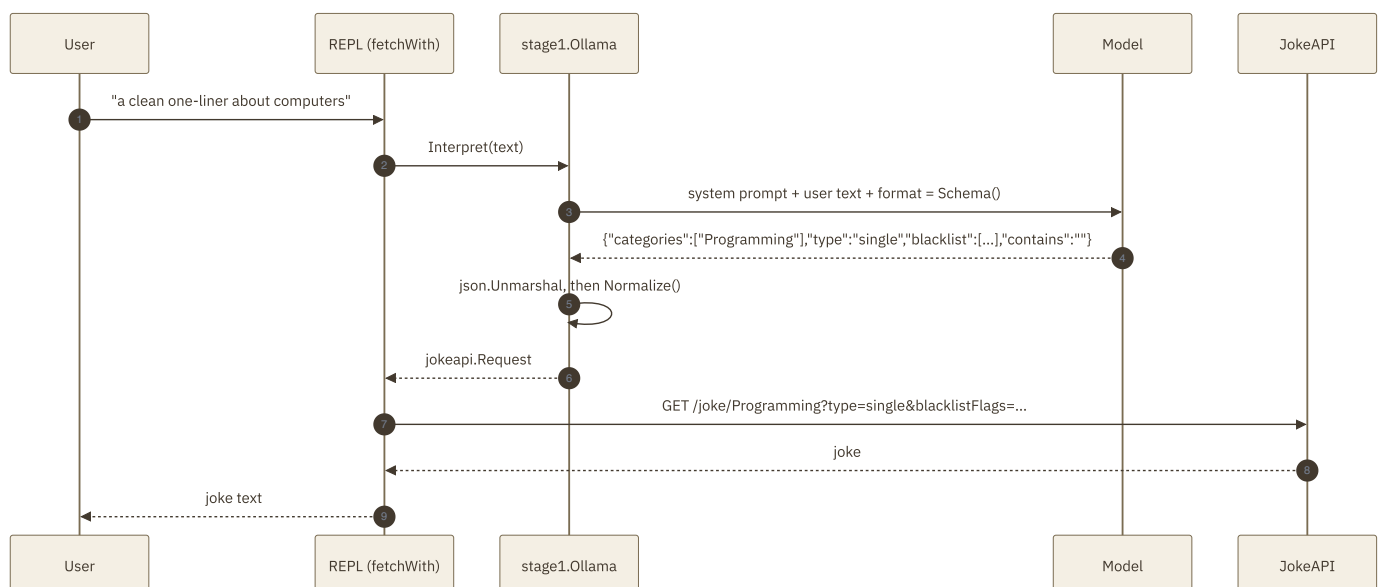
TRY IT

```
go run ./cmd/jokes -stage 0 -trace
> keep it family friendly      # watch the empty blacklist in the trace
```

● STAGE 1 · INTERNAL/STAGE1 · **BUILT**

Structured output: the model as a parser

One LLM call, and the model's only job is to fill in a form. We send a JSON schema in Ollama's `format` field. Ollama compiles it into a grammar, so the model can only emit tokens that keep the JSON valid. Our code decodes the reply, calls `Normalize()`, and does everything else itself.



The model is called once. Your code owns every other step.

The schema is the contract

`jokeapi.Schema()` lists every property as required and forbids extras, which strict structured-output modes demand. "No preference" is spelled `"any"`, `[]` or `""`.

```
// what the model must produce
{
  "categories": ["Programming"], // enum of 6 categories
  "type":       "single",        // "any" | "single" | "twopart"
  "blacklist":  ["nsfw", "racist"], // enum of 6 flags
  "contains":   ""               // literal substring, usually empty
}
```

Why Normalize() still exists

Even under a schema, small models produce casing differences, duplicates and `"any"`.

`Normalize()` matches values case-insensitively against the known lists, drops unknowns and duplicates, and maps `"any"` to empty. Model output is data to be cleaned, never trusted input.

Two backends, one prompt

Ollama (`ollama.go`)

Local model. Thinking is turned off because a form-filling task doesn't need it and reasoning costs seconds. Fails loudly if `done_reason` is `length`.

Claude API (`claude.go`)

Same system prompt and schema through the Anthropic SDK, for side-by-side comparison. Needs a Console API key, which a Pro subscription doesn't include.

LESSON

With a schema, the model cannot return anything your code can't decode. The LLM becomes a typed function: text in, struct out.

Known weakness

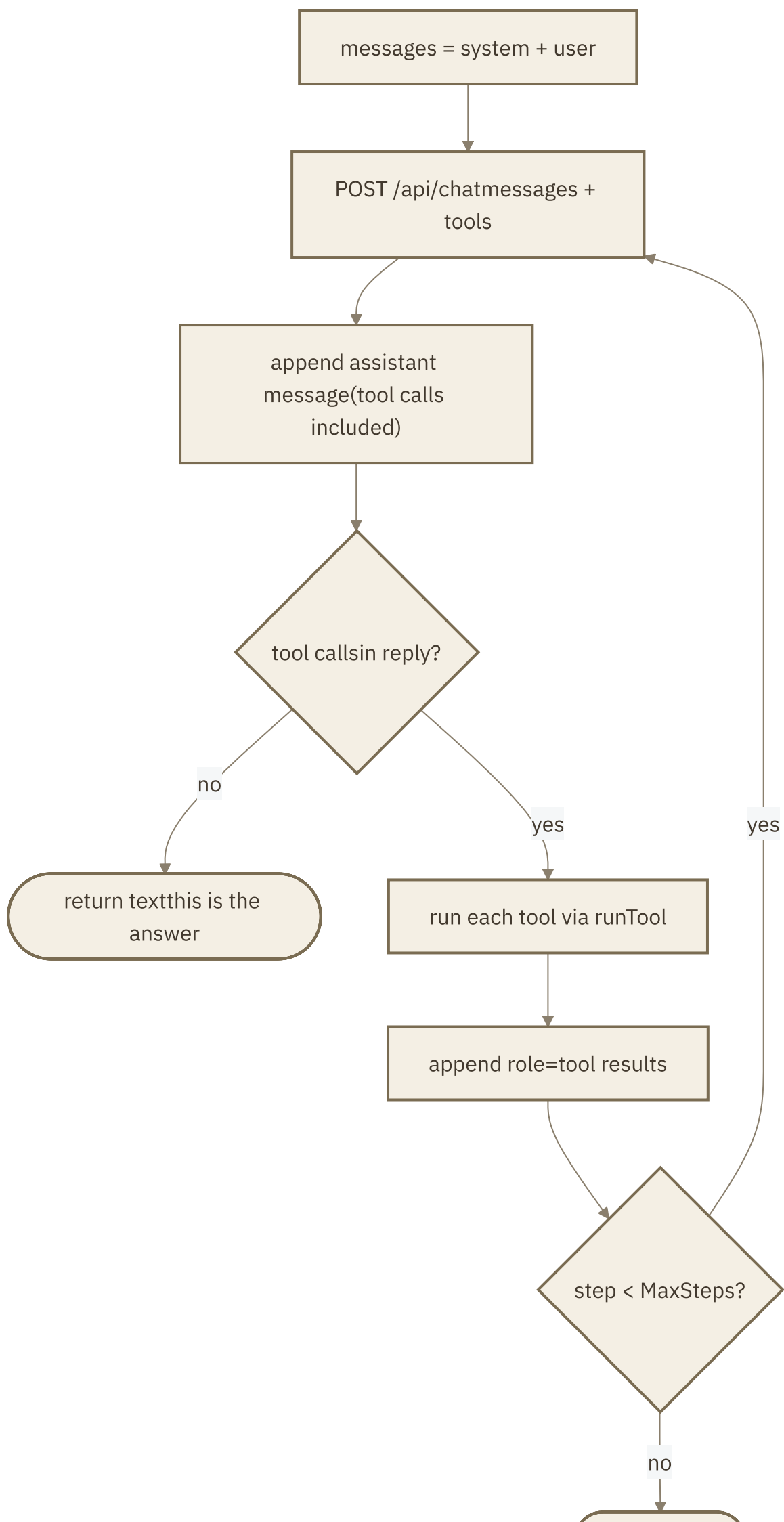
"a joke about the chicago cubs, it can be dirty but not racist" becomes `contains: "cubs"`, which matches nothing, and stage 1 has no way to recover. That is the opening for stage 2.

TRY IT

```
go run ./cmd/jokes -stage 1 -trace
> nothing about politics or religion # compare with -stage 0
```

Tool calling: the agent loop

In stage 1 your code decided what happens after the model answers. Now the model decides. We describe a `get_joke` tool, and the model chooses when to call it, with what arguments, and what to do with the result. The whole agent is one `for` loop in `Respond`.



Frameworks and SDK tool runners are conveniences over exactly this loop.

A tool has two halves

What the model sees

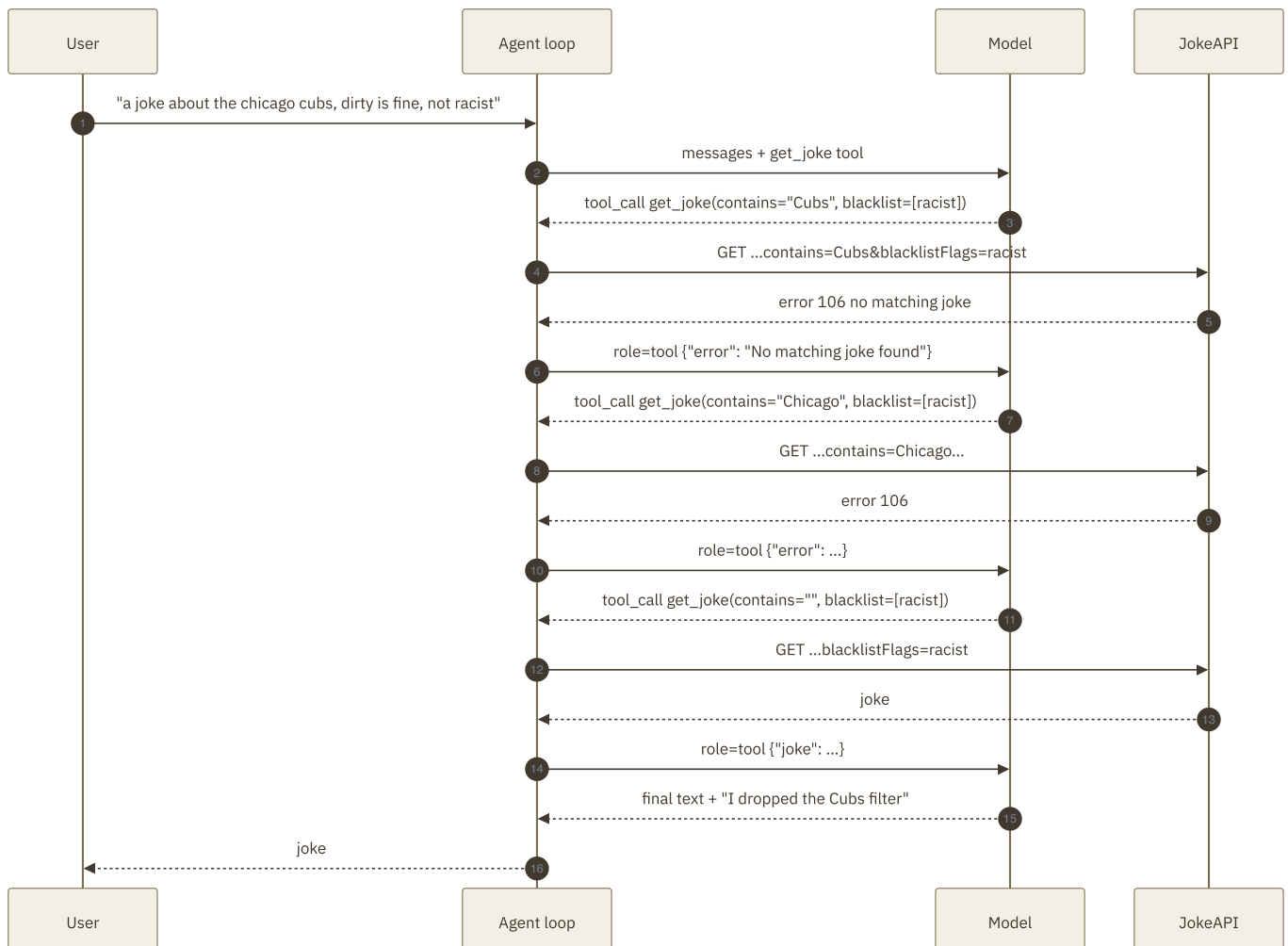
A name, a natural-language description and a JSON schema for parameters. Stage 2 reuses `jokeapi.Schema()` as the parameter schema, so structured output and tool input share one definition.

What your code does

`runTool` decodes the arguments, runs `Normalize()`, calls `Fetch`, and returns JSON text. Unknown tool names and bad arguments also come back as text.

Errors are results, not exceptions

If JokeAPI says "No matching joke found", the loop does not abort. The error goes back as a tool message, and the model reads it and tries again. That is how retry behaviour appears without writing a retry loop.



The retry sequence from the README. Notice `racist` stays on the blacklist through every attempt.

Guard rails in the prompt and in code

GUARD	WHERE	WHY
MaxSteps = 6	code	A confused model could loop on tool calls forever, and each step costs seconds locally.
"Never loosen the blacklist"	system prompt	Retrying is allowed, weakening the user's safety limits is not.
"After 3 failed calls, give up"	system prompt	Gives the model a graceful way out before MaxSteps hits.
done_reason = length	code	A truncated reply is reported as an error, not shown as an answer.
<code>-think</code> flag	code	Optional reasoning before acting. Smarter, slower. Compare the step counts.

LESSON

An agent is a loop where the model's output can be an action. The assistant message with its tool calls must go back into history, or the model forgets what it already asked for.

TRY IT

```
go run ./cmd/jokes -stage 2 -trace
> a joke about the chicago cubs, it can be dirty but not racist
# then repeat with -think and compare the number of steps
```

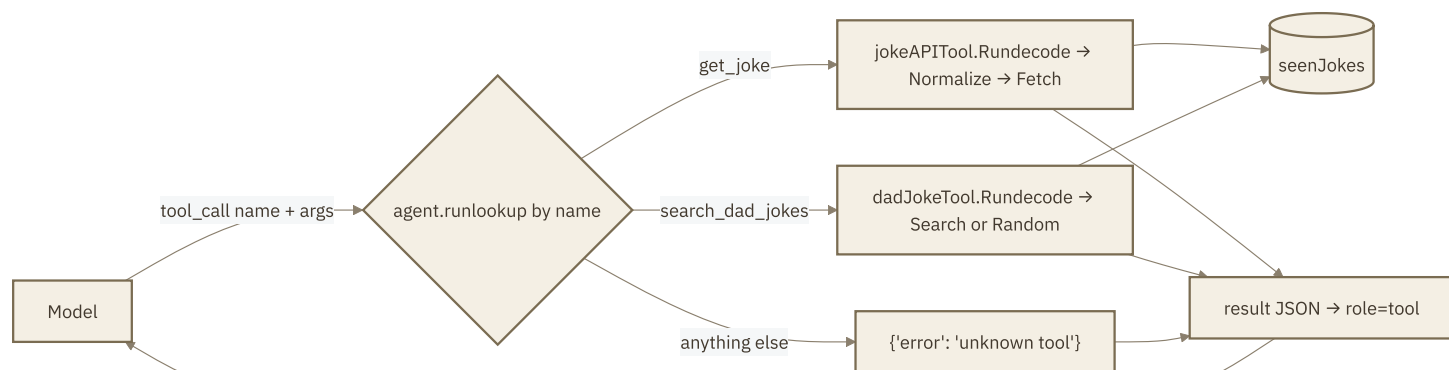
Many tools, memory and a limited context window

Stage 3 adds three things at once. They belong together because each one raises a question the next one answers.

1. A tool registry and tool choice

There are now two tools. `get_joke` wraps JokeAPI and supports categories and content blacklists. `search_dad_jokes` wraps icanhazdadjoke and takes one search word. Each is a

`Tool{Def, Run}` : `Def` is what the model sees, `Run` is what your code does. The agent keeps a map from name to tool.



The model picks a tool from the descriptions alone, so the descriptions are part of the prompt.

2. Memory is re-sending the transcript

The model has no memory. The agent stores `history` (without the system prompt) and sends it again on every call. That single change makes "another one", "explain that one" and "keep it clean from now on" work. Some turns need no tool at all, and the model has to notice that.

Memory in the model

The conversation text. It is flexible but fuzzy, and it is the first thing lost when trimmed.

Memory in your code

`seenJokes` records which joke IDs were already told and makes tools re-fetch (up to 3 times) on a repeat. It is exact and survives trimming.

3. Trimming the context window

History grows every turn but `num_ctx` stays at 8192. Before each user turn, `trim()` drops the oldest whole turn while either condition holds:

- the number of turns has reached `MaxTurns` (10), or
- the last prompt used more than 3/4 of the context window (`NumCtx * 3 / 4` , so 6144 tokens).

It cuts whole turns only. Dropping half a turn would leave a tool result with no call to answer.

Before trim



After trim

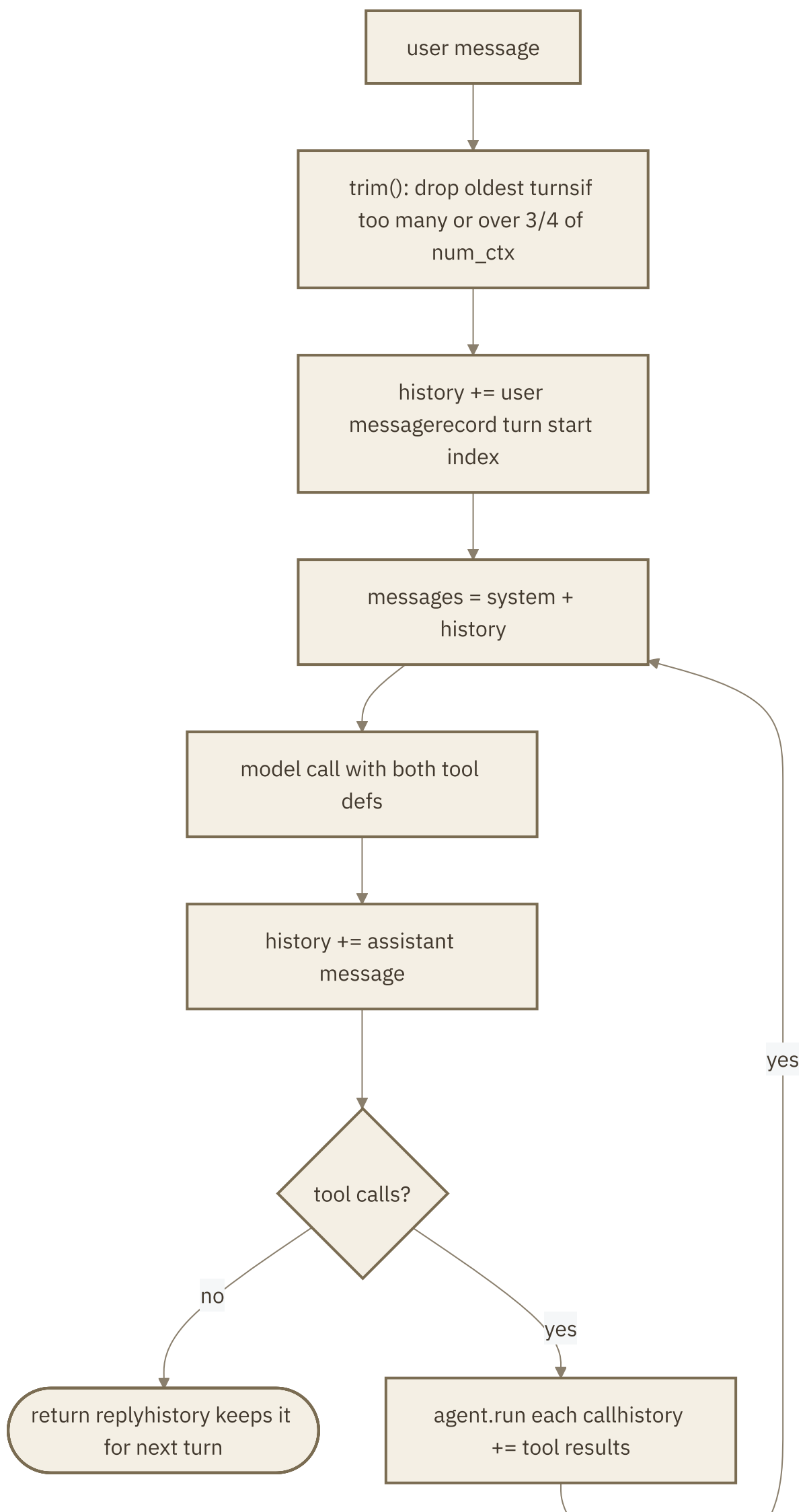


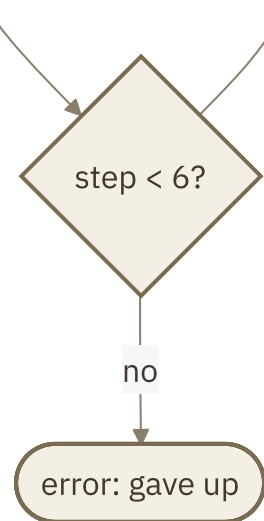
The restriction the user gave in turn 1 disappears with the turn. The model now has no idea it ever existed.

The trade-off this stage exists to show

A restriction given early ("nothing racist") can be trimmed away. Real systems summarize old turns or pin standing instructions in the system prompt. Run with `-ctx 2048` to force trimming early and watch it happen. A second known weakness: after an "explain it" turn, the model sometimes explains later jokes unasked, and sometimes wrongly.

One user message, end to end





Same loop as stage 2. The differences are where history lives and the trim step in front.

A conversation to try

```
go run ./cmd/jokes -stage 3 -trace
> a joke about dogs           # likely search_dad_jokes("dog")
> another one                 # memory: same tool, new joke
> explain that one            # no tool needed
> from now on nothing political. a programming joke # likely get_joke, blacklist=[political]
> /reset                      # forget history and seenJokes
```

LESSON

Tools, memory and context budget are one system. Whatever you cannot afford to lose should live in code or in the system prompt, not in the oldest message.

● STAGE 3B · INTERNAL/STAGE3LC · **BUILT**

The same agent in langchaingo

Stage 3 again, built from `agents.Executor`, an OpenAI-functions agent and a window-buffer memory. The loop, the history slice and `trim()` disappear from your code. The point is to see what you give up for that, so each hand-written piece is paired with what it turned into.

HAND-WRITTEN IN STAGE 3

IN LANGCHAINGO

WHAT WE FOUND

Ollama HTTP structs

`openai.LLM` pointed at
Ollama's `/v1`

The native Ollama client drops tool definitions, so tools only work through the OpenAI-compatible endpoint. That endpoint has no `num_ctx` or `keep_alive`,

so this stage cannot apply the repo's memory limits.

`Tool{Def, Run}`

`tools.Tool : Name,`
`Description, Call(string)`

One free-text string. The agent advertises every tool as `{"__arg1": string}`. The typed schema becomes prose in the description, the model writes JSON inside a string, and we parse it back. Stage 1's lesson has to be learned again.

`for step := ...`

`agents.Executor`

Step cap is `MaxIterations`. A Go error from a tool aborts the whole run, so our tools return errors as text instead.

`history + trim()`

`ConversationWindowBuffer`

Keeps the user and AI text only, not tool calls or results. Executor inputs must be strings, so the history is flattened to "Human: ... AI: ..." and pasted into the system prompt. It windows by exchange count, with no token budget.

`-trace`

`callbacks.Handler`

Works. Prompts and tool results are available, but token counts hide inside `GenerationInfo`.

`num_predict`

chain call option
`WithMaxTokens`

Sent as `max_completion_tokens`, which servers that only imitate the OpenAI API may ignore. Switching to `max_tokens` needs a call option the executor can't pass, so a small wrapper around the model adds it.

LESSON

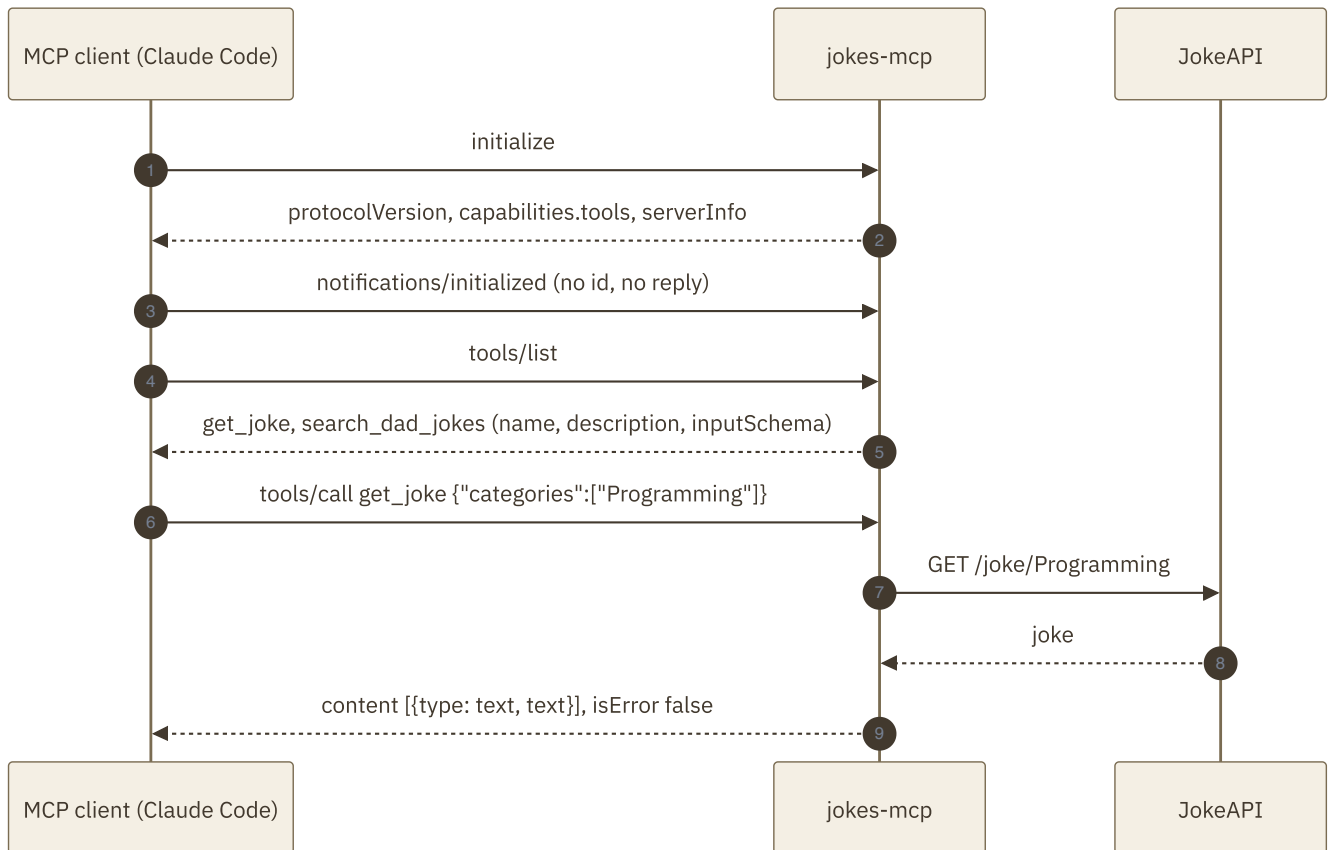
A framework removes code, not decisions. String-only tool input, flattened history and aborting on tool errors are decisions you inherit. The stage 3 and 3b files are about the same length (276 and 274 lines).

TRY IT

```
go run ./cmd/jokes -stage 3b -trace
> a programming joke      # compare the trace with stage 3's, step by step
```

MCP: the tools leave the agent

In stages 2 and 3 the tools lived inside our agent, wired to one model API. The Model Context Protocol moves them behind a standard interface. The client owns the model and the loop. The server only describes tools and runs them, so our agent loop isn't needed at all, and Claude Code can use the same jokes.



The whole protocol this server needs: JSON-RPC 2.0, one JSON message per line on stdin and stdout.

Hand-written, no SDK

`server.go` is about 160 lines: read a line, decode, switch on `method`, write a reply. A request without an `id` is a notification and gets no answer.

stdout belongs to the protocol

Any stray print would corrupt the stream, so `-trace` logs every JSON-RPC message to stderr instead.

Tool failures are results

"No matching joke" returns `isError: true` inside a normal reply, not a JSON-RPC error. The calling model sees it and can retry, same as stages 2 and 3.

No dedupe, by design

A server doesn't know which conversation a call belongs to, so "don't repeat a joke" is the client's job. Stage 6 moves it into its loop.

Unknown tools and bad params are real protocol errors.

LESSON

MCP separates "what can be done" from "who decides". Everything about choosing tools, retrying and remembering moves to the client.

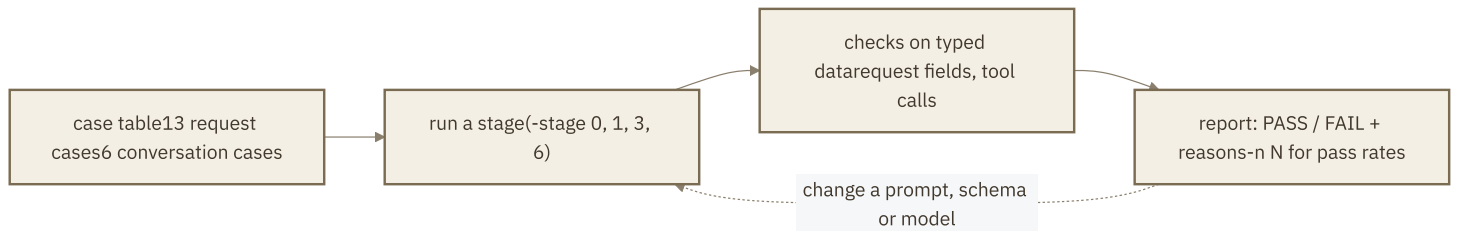
TRY IT

```
claude mcp add jokes -- go run ./cmd/jokes-mcp
# then ask Claude Code: "a clean dad joke about dogs"
go run ./cmd/jokes-mcp -trace    # watch the JSON-RPC on stderr
```

● STAGE 5 · INTERNAL/EVALS · CMD/JOKES-EVAL · **BUILT**

Evals: from "it seemed to work" to a number

A table of cases, each with plain Go checks that return "" for a pass or a reason for a fail. There are two kinds. Request cases (stages 0 and 1) map one prompt to the typed `jokeapi.Request` the interpreter produced. Conversation cases (stages 3 and 6) script several user messages and check which tools the agent called, with what arguments.



Checks look at requests and tool calls, not at the wording of the reply. Agents expose an `OnToo1` hook so the runner can see the calls.

First scores

STAGE	CASES PASSED	WHAT FAILED
0 Keywords	7 of 13	"clean" and "family friendly" don't expand to a full blacklist, "politics or religion" is missed, no topic word is extracted.

1 Structured, qwen3.5:9b-mlx 8 of 13

"clean" and "family friendly" still don't expand fully (the known weakness), "dark or punny" lost `Pun`, "a spooky joke" became `Misc`, "clean christmas" lost the category.

6 Agent, qwen3.5:9b-mlx 5 of 6

`no-unasked-explanations`: after "explain that joke", the reply to "another one" was 361 characters for a 49-character joke, so it explained the joke unasked (the known weakness). Tool choice, `clean` expansion and a restriction carried across turns all passed.

Each is a single run on one day. Model output varies, so use `-n 3` or more and read the pass rate. The stage 0 score is exact, since it has no model.

The known weaknesses are cases on purpose

`clean` guards the full-blacklist expansion. `no-unasked-explanations` guards stage 3 explaining later jokes after an "explain it" turn. They should fail until a prompt or schema change fixes them, and then they stop it coming back.

LESSON

The first surprise is usually the baseline. A 9B model beat `strings.Contains` by one case, which is the argument for measuring before and after every prompt change.

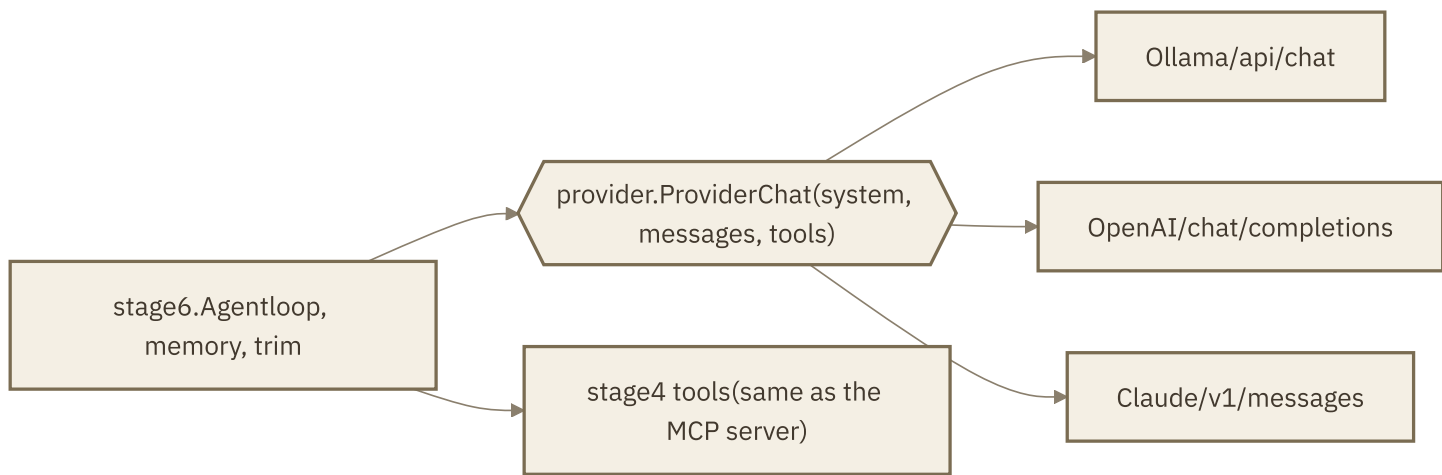
TRY IT

```
make eval # stage 0, no LLM
go run ./cmd/jokes-eval -stage 1 -n 3 # pass rates over 3 runs
go run ./cmd/jokes-eval -stage 3 -run restriction # one conversation case
```

● STAGE 6 · INTERNAL/PROVIDER · INTERNAL/STAGE6 · **BUILT**

Provider switching: one loop, any model

Stage 6 is the stage 3 agent over a small `provider.Provider` interface: system prompt, messages and tools in, one message out. Ollama, OpenAI and Claude each get an adapter. Switching is a flag, so the same eval can score all three.



Each adapter is plain `net/http` with structs that mirror its own wire format.

What the adapters hide

	OLLAMA	OPENAI	CLAUDE
System prompt	a <code>system</code> message	a <code>system</code> message	a top-level <code>system</code> field
Tool call arguments	a JSON object	a JSON <i>string</i> inside the JSON	a JSON object (<code>input</code>)
Tool result	<code>tool</code> message with <code>tool_name</code>	<code>tool</code> message with <code>tool_call_id</code>	<code>tool_result</code> block in a <i>user</i> message; all results of one turn in one message
Call IDs	none, the agent invents them	required to pair results	required to pair results
Output cap	<code>num_predict</code>	<code>max_tokens</code>	<code>max_tokens</code> , required

Tools are stage 4's

The agent wraps the same tool implementation the MCP server exposes. They only return text, so the "already told" check moved from the tools into the loop.

Token budget is optional

Ollama gets 3/4 of `num_ctx` , as before. Hosted models have far larger windows, so only the turn limit applies.

Not yet run live

The OpenAI and Claude adapters are tested against fake servers that check each vendor's request shape. They have not been run against the real

APIs, because there are no keys on the dev machine.

LESSON

The loop is the same everywhere. What differs between providers is the plumbing around it, and then the behaviour: run the same eval on each and compare.

TRY IT

```
go run ./cmd/jokes -stage 6 -provider ollama -trace
ANTHROPIC_API_KEY=... go run ./cmd/jokes -stage 6 -provider claude
OPENAI_API_KEY=... go run ./cmd/jokes-eval -stage 6 -provider openai -n 3
# OPENAI_BASE_URL points the openai adapter at any compatible server
```

WHAT'S LEFT

Ideas if you keep going

Every planned stage is built. These are the loose ends the work turned up.

- Fix the failing eval cases by changing prompts, then re-run with `-n 3` to see which fixes hold.
- Run the stage 6 eval on Claude and OpenAI once API keys exist, and record the pass rates next to the local model's.
- Give stage 3b a way to set `num_ctx`, for example through a Modelfile, so it respects the memory limits.
- Add a resource or a prompt to the MCP server, not just tools.
- Pin standing restrictions in the system prompt so trimming can't erase them.

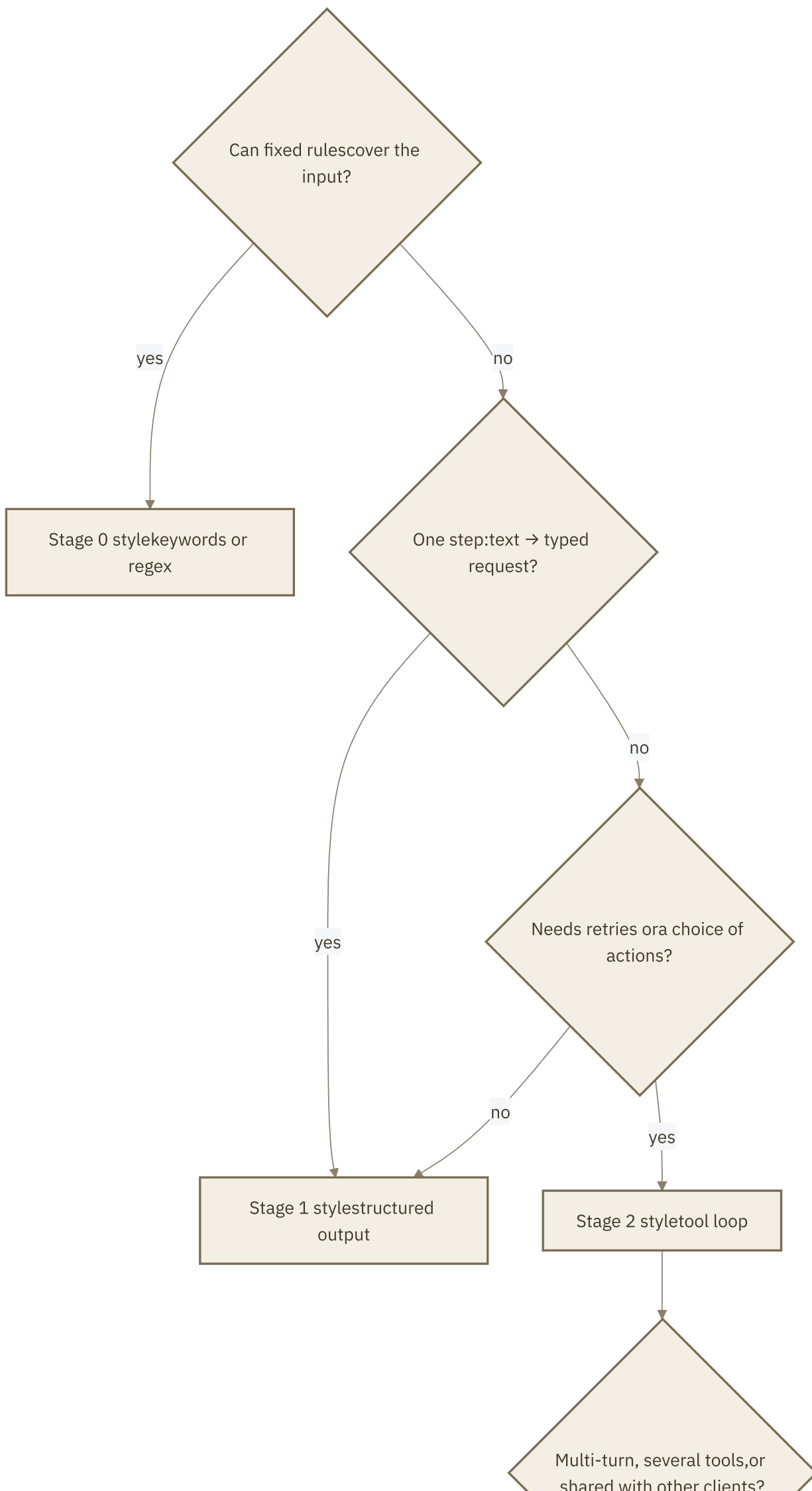
BEYOND JOKES

Applying this to other projects

Jokes are a stand-in. The structure carries over to anything where a person asks in plain language and a typed API does the work. Swap the domain and keep the skeleton.

<code>jokeapi.Request</code>	Your typed request struct	<code>TicketQuery{Status, Priority, Assignee, Since}</code>
<code>Schema()</code>	JSON schema generated from that struct	Enums for status and priority
<code>Normalize()</code>	Validation of model output	Reject unknown assignees, clamp date ranges
<code>get_joke</code> tool	One tool per API capability	<code>search_tickets</code> , <code>get_ticket</code> , <code>add_comment</code>
"No matching joke" as a tool result	Errors the model can act on	"0 tickets match. Try a wider date range."
Blacklist never loosened	Hard constraints in prompt <i>and</i> code	Read-only mode, no deleting, scope to one project
<code>seenJokes</code>	Exact state kept in code	Tickets already shown, IDs already modified
<code>trim()</code>	Context policy	Summarize old turns, pin the user's project and role
<code>-trace</code>	Observability	Log every prompt, tool call, result and token count

A decision guide



Start at the simplest stage that works. Each step down adds cost, latency and ways to fail.

Habits worth taking with you

Typed boundaries

Models write structs, not strings. Decode, normalize, then act.

Tool errors go to the model

Return failures as results so it can retry, instead of aborting the turn.

Cap every loop

Step limits, retry limits and output limits are cheap insurance.

Trace everything

Most of the learning, and most of the debugging, is reading the prompts and tool results.

Fake the model in tests

A scripted `httptest` server replays a conversation, so loop logic is tested without loading a model.

Measure with evals

Turn known failures into test cases before changing prompts.

Running it

yes

yes

```
ollama pull qwen3.5:9b-m1x
make test                # offline unit tests
make live                # also hits the real joke APIs
go run ./cmd/jokes -stage 3 -trace  # needs local Ollama
make eval                # stage 0 baseline, no LLM
go run ./cmd/jokes-mcp   # MCP server on stdio
```

FLAG	DEFAULT	CONTROLS
<code>-ctx</code>	8192	Context window. Large defaults can push a 16 GB Mac into swap. Not settable in stage 3b.
<code>-keep-alive</code>	2m	How long the model stays loaded when idle.
<code>-think</code>	off	Reasoning before acting, stages 2, 3 and 6.
<code>-provider</code>	ollama	Stage 6 only: ollama, openai or claude. Keys come from <code>OPENAI_API_KEY</code> and <code>ANTHROPIC_API_KEY</code> .

`-trace`

off

Print prompts, model output, tool calls and results to stderr.